title: "幫我把整段完整的 HTML 程式碼重新組合一次"

exported: 2026-03-18 04:08:07

url: https://gemini.google.com/app/c890f0fb2770b20b?hl=zh-TW

messages: 4

# 幫我把整段完整的 HTML 程式碼重新組合一次

## You

可以幫我把最佳路徑標示出來嗎

## Gemini

這是一個很棒的主意！在強化學習的視覺化中，除了顯示每個格子的策略（方向），直接將從起點到終點的「最優路徑」連貫地標示出來，能更直觀地看出演算法的成果。

既然你目前的設計已經有了**價值矩陣 (Value Matrix)** 與 **策略矩陣 (Policy Matrix)**，我們可以透過修改 `index.html` 中的 `calc()` 函式，增加一個尋路邏輯：從起點開始，根據 `policy` 的方向一路走到終點，並給這些格子加上特殊的樣式（例如黃色背景）。

以下是修改建議：

### 1. 修改 CSS 樣式

在 `<style>` 區塊中新增一個 `path` 類別，用來突顯路徑。

```
/* 新增路徑樣式 */
.path {
    background: #fff59d !important; /* 明亮的黃色 */
    border: 2px solid #fbc02d;
    box-shadow: inset 0 0 10px rgba(0,0,0,0.1);
}
```

---

### 2. 更新 `calc()` 函式與新增路徑邏輯

我們在 `calc()` 執行完畢後，增加一段邏輯來追蹤路徑：

```
function calc() {
    statusBox.innerText = "計算中...";
    const result = valueIteration(n, start, end, obs);
    const arrowMap = {'U':'↑','D':'↓','L':'←','R':'→'};
    const actions = { 'U': [-1, 0], 'D': [1, 0], 'L': [0, -1], 'R': [0, 1] };

    // 1. 清除舊的路徑樣式 (若有重複計算需求)
    document.querySelectorAll('.cell').forEach(c => c.classList.remove('path'));

    // 2. 渲染數值與箭頭
    for(let r=0; r<n; r++) {
        for(let c=0; c<n; c++) {
            const vCell = document.getElementById(`vGrid-${r}-${c}`);
            const pCell = document.getElementById(`pGrid-${r}-${c}`);

            if(!vCell.classList.contains('start') && !vCell.classList.contains('end') && !vCell.classList.contains('obstacle')) {
                vCell.innerHTML = `<span class="val-text">${result.V[r][c].toFixed(2)}</span>`;
                pCell.innerHTML = `<span class="policy-text">${arrowMap[result.policy[r][c]]}</span>`;
            }
        }
    }

    // 3. 標示最佳路徑
    let currR = start[0];
    let currC = start[1];
    let step = 0;
    const maxSteps = n * n; // 防止無限迴圈

    while (step < maxSteps) {
        // 如果走到終點，停止
        if (currR === end[0] && currC === end[1]) break;

        // 標示當前路徑 (起點除外，保持綠色比較好辨識)
        if (!(currR === start[0] && currC === start[1])) {
            document.getElementById(`vGrid-${currR}-${currC}`).classList.add('path');
            document.getElementById(`pGrid-${currR}-${currC}`).classList.add('path');
        }

        // 根據 Policy 移向下一格
        let move = result.policy[currR][currC];
        let [dr, dc] = actions[move];
        currR += dr;
```

```
        currC += dc;
        step++;
    }

    statusBox.innerText = "計算完成！最佳路徑已用黃色標示。";
}
```

---

### 運作原理

1. **路徑追蹤**：這段程式碼會從 `start` 座標出發。
2. **查詢策略**：它會查看 `result.policy` 在該座標給出的建議方向（如 'U' 或 'R'）。
3. **移動座標**：根據方向移動到下一個格子 `currR, currC`。
4. **視覺標示**：給該格子的 DOM 元素加上 `.path` 類別，使其變成黃色。
5. **終止條件**：直到座標等於 `end` 或是超過最大步數（防止因策略未收斂導致的死迴圈）為止。

這樣一來，你的 GridWorld 模擬器就能清楚地展現出從起點到終點的那條「最優路線」了！

需要我幫你把整段完整的 HTML 程式碼重新組合一次嗎？

---

## You

幫我把整段完整的 HTML 程式碼重新組合一次

---

## Gemini

這是一份完整的 `index.html` 程式碼。我已經將 **CSS 樣式**、**路徑追蹤邏輯**以及** UI 更新機制**全部整合在一起。

你可以直接替換掉原本的 `index.html` 檔案：

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
    <meta charset="UTF-8">
    <title>DRL GridWorld 最佳路徑模擬器</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; padding: 20px; background-color: #f5f7f9; }
        .grid { display: grid; gap: 4px; background: #ccc; border: 2px solid #999; padding: 4px; border-radius: 4px; }
        .cell {
            width: 60px; height: 60px; background: #fff;
            display: flex; flex-direction: column; align-items: center; justify-content: center;
            cursor: pointer; font-size: 14px; transition: all 0.2s; border-radius: 2px;
            position: relative;
        }
        .cell:hover { background: #e0e0e0; }
        .start { background: #4CAF50 !important; color: white; font-weight: bold; }
        .end { background: #f44336 !important; color: white; font-weight: bold; }
        .obstacle { background: #607d8b !important; color: white; }

        /* 最佳路徑樣式 */
        .path {
            background: #fff59d !important; /* 明亮的黃色 */
            border: 2px solid #fbc02d;
            box-shadow: inset 0 0 10px rgba(0,0,0,0.1);
        }

        .container { display: flex; gap: 40px; margin-top: 20px; flex-wrap: wrap; }
        .label { margin-bottom: 8px; font-weight: bold; color: #333; }
        #status-box {
            padding: 15px; background: #e3f2fd; border-left: 5px solid #2196f3;
            margin-bottom: 20px; font-weight: bold; color: #0d47a1; border-radius: 4px;
        }
        .val-text { font-size: 12px; color: #1565c0; }
        .policy-text { font-size: 24px; font-weight: bold; color: #2e7d32; }
        input[type="number"] { padding: 5px; border: 1px solid #ccc; border-radius: 4px; width: 50px; }
        button { padding: 6px 15px; cursor: pointer; background: #2196f3; color: white; border: none; border-radius: 4px; transition: 0.2
        button:disabled { background: #bdbdbd; cursor: not-allowed; }
        button:hover:not(:disabled) { background: #1976d2; }
    </style>
</head>
<body>
    <h2>DRL GridWorld 最佳路徑模擬器 (含路徑標示)</h2>

    <div id="status-box">請設定網格尺寸並點擊「生成網格」開始。</div>

    <div style="margin-bottom: 20px;">
        網格尺寸 (n x n): <input type="number" id="n" value="5" min="5" max="9">
        <button onclick="gen()">生成網格</button>
        <button id="calcBtn" onclick="calc()" disabled>計算最佳路徑</button>
    </div>

    <div class="container">
        <div>
            <div class="label">價值矩陣 (Value Matrix)</div>
            <div id="vGrid" class="grid"></div>
        </div>
```

```
    <div>
        <div class="label">策略矩陣 (Policy Matrix)</div>
        <div id="pGrid" class="grid"></div>
    </div>
</div>

<script>
    let n, start, end, obs = [];
    const statusBox = document.getElementById('status-box');
    const calcBtn = document.getElementById('calcBtn');

    // 生成網格
    function gen() {
        n = parseInt(document.getElementById('n').value);
        start = null; end = null; obs = [];
        calcBtn.disabled = true;
        statusBox.innerText = "第一步：請點選一個格子作為「起點」(綠色)。";

        ['vGrid', 'pGrid'].forEach(id => {
            let el = document.getElementById(id);
            el.style.gridTemplateColumns = `repeat(${n}, 60px)`;
            el.innerHTML = '';
            for(let i=0; i<n*n; i++) {
                let r = Math.floor(i/n), c = i%n;
                let cell = document.createElement('div');
                cell.className = 'cell';
                cell.id = `${id}-${r}-${c}`;
                cell.innerText = i + 1;
                cell.onclick = () => handleCellClick(r, c);
                el.appendChild(cell);
            }
        });
    }

    // 處理點擊事件
    function handleCellClick(r, c) {
        if(!start) {
            start = [r,c];
            updateUI(r, c, 'start', '起點');
            statusBox.innerText = "第二步：請點選一個格子作為「終點」(紅色)。";
        } else if(!end) {
            if(r === start[0] && c === start[1]) return;
            end = [r,c];
            updateUI(r, c, 'end', '終點');
            statusBox.innerText = `第三步：請點選 ${n-2} 個「障礙物」(灰色)。`;
        } else if(obs.length < n-2) {
            if((r===start[0] && c===start[1]) || (r===end[0] && c===end[1])) return;
            if(obs.some(o => o[0] === r && o[1] === c)) return;
            obs.push([r,c]);
            updateUI(r, c, 'obstacle', '阻礙');
            let rem = (n-2) - obs.length;
            if(rem > 0) statusBox.innerText = `還剩 ${rem} 個障礙物需要設定。`;
            else {
                statusBox.innerText = "設定完成！請點擊「計算最佳路徑」。";
                calcBtn.disabled = false;
            }
        }
    }

    function updateUI(r, c, cls, text) {
        ['vGrid', 'pGrid'].forEach(id => {
            const cell = document.getElementById(`${id}-${r}-${c}`);
            cell.classList.add(cls);
            cell.innerText = text;
        });
    }

    // 核心 Value Iteration 演算法
    function valueIteration(n, start, end, obstacles) {
        let V = Array.from({ length: n }, () => Array(n).fill(0));
        let policy = Array.from({ length: n }, () => Array(n).fill('U'));
        const gamma = 0.9;
        const actions = { 'U': [-1, 0], 'D': [1, 0], 'L': [0, -1], 'R': [0, 1] };

        const isObs = (r, c) => obstacles.some(o => o[0] === r && o[1] === c);
        const isEnd = (r, c) => r === end[0] && c === end[1];

        for (let i = 0; i < 100; i++) {
            let new_V = V.map(row => [...row]);
            for (let r = 0; r < n; r++) {
                for (let c = 0; c < n; c++) {
                    if (isEnd(r, c) || isObs(r, c)) continue;
                    let actionValues = [];
                    for (let [dr, dc] of Object.values(actions)) {
                        let nr = r + dr, nc = c + dc;
                        if (nr >= 0 && nr < n && nc >= 0 && nc < n) {
                            if (isObs(nr, nc)) actionValues.push(-1 + gamma * V[r][c]);
                            else if (isEnd(nr, nc)) actionValues.push(10 + gamma * 0);
                            else actionValues.push(-0.1 + gamma * V[nr][nc]);
                        } else {
```

```
                        actionValues.push(-1 + gamma * V[r][c]);
                    }
                }
                new_V[r][c] = Math.max(...actionValues);
            }
        }
        V = new_V;
    }

    for (let r = 0; r < n; r++) {
        for (let c = 0; c < n; c++) {
            if (isEnd(r, c) || isObs(r, c)) continue;
            let bestAction = 'U', maxVal = -Infinity;
            for (let [name, [dr, dc]] of Object.entries(actions)) {
                let nr = r + dr, nc = c + dc, val;
                if (nr >= 0 && nr < n && nc >= 0 && nc < n && !isObs(nr, nc)) {
                    val = (isEnd(nr, nc) ? 10 : -0.1) + gamma * V[nr][nc];
                } else {
                    val = -1 + gamma * V[r][c];
                }
                if (val > maxVal) { maxVal = val; bestAction = name; }
            }
            policy[r][c] = bestAction;
        }
    }
    return { V, policy };
}

function calc() {
    statusBox.innerText = "計算中...";
    const result = valueIteration(n, start, end, obs);
    const arrowMap = {'U':'↑','D':'↓','L':'←','R':'→'};
    const actions = { 'U': [-1, 0], 'D': [1, 0], 'L': [0, -1], 'R': [0, 1] };

    // 清除舊的路徑標示
    document.querySelectorAll('.cell').forEach(c => c.classList.remove('path'));

    // 渲染 V 與 Policy
    for(let r=0; r<n; r++) {
        for(let c=0; c<n; c++) {
            const vCell = document.getElementById(`vGrid-${r}-${c}`);
            const pCell = document.getElementById(`pGrid-${r}-${c}`);

            if(!vCell.classList.contains('start') && !vCell.classList.contains('end') && !vCell.classList.contains('obstacle')) {
                vCell.innerHTML = `<span class="val-text">${result.V[r][c].toFixed(2)}</span>`;
                pCell.innerHTML = `<span class="policy-text">${arrowMap[result.policy[r][c]]}</span>`;
            }
        }
    }

    // 標示最佳路徑
    let currR = start[0];
    let currC = start[1];
    let step = 0;
    const maxSteps = n * n;

    while (step < maxSteps) {
        if (currR === end[0] && currC === end[1]) break;

        // 標示當前路徑 (跳過起點)
        if (!(currR === start[0] && currC === start[1])) {
            document.getElementById(`vGrid-${currR}-${currC}`).classList.add('path');
            document.getElementById(`pGrid-${currR}-${currC}`).classList.add('path');
        }

        let move = result.policy[currR][currC];
        let [dr, dc] = actions[move];
        currR += dr;
        currC += dc;
        step++;
    }

    statusBox.innerText = "計算完成！最佳路徑已用黃色標示。";
}

gen();
</script>
</body>
</html>
```